CYS-262L — Information Security Lab

Week 5: Password Storage, Password Policies and MFA (TOTP) on Linux

Student Name: Muhamad Anees Ahmad

Registration No.: 2025(S)-CYS-63

Section : B

Practical lab report with command explanations, observations, and screenshot spaces.

Purpose of the Lab

The purpose of this lab was to examine how Linux stores password information, review file permissions, apply password-quality and expiry rules, and test time-based one-time passwords (TOTP) through PAM. The exercises also demonstrate why salts, restricted permissions, MFA, and a recovery plan are important.

Environment and Safety Notes

Environment: Linux virtual machine running in VMware. Before changing password or PAM settings, a VM snapshot should be created. The commands below are intended for an isolated lab VM and should be run only where the student has permission. Password hashes, authenticator secrets, QR codes, and emergency codes must not be included in screenshots.

Note: Results can differ by Linux distribution, PAM configuration, and installed package versions. In the Observation fields, enter what actually happened on your VM instead of claiming an expected result occurred.

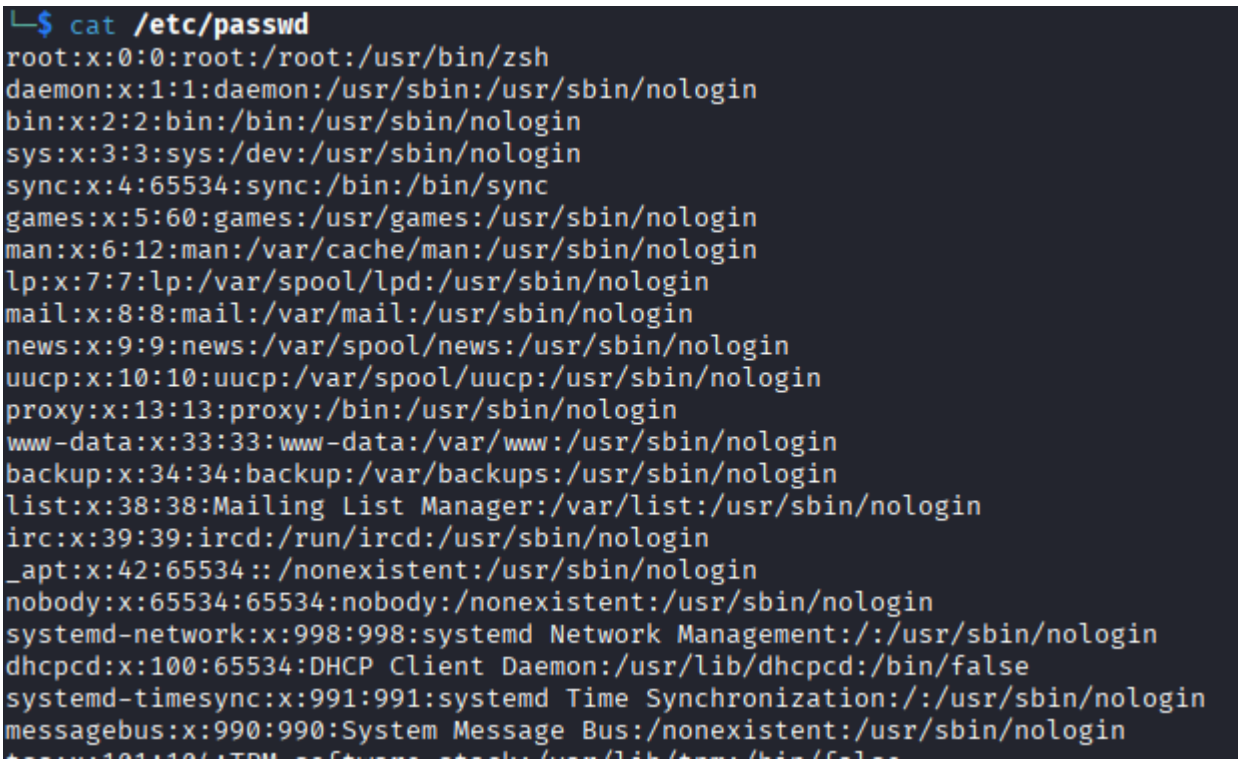
Task A — Authentication and Password Storage

A1. View Linux account records

The /etc/passwd file contains account metadata such as username, UID, GID, home directory, and login shell. It normally does not contain the actual password hash.

```
cat /etc/passwd
```

Expected result: Account entries are printed, one per line.

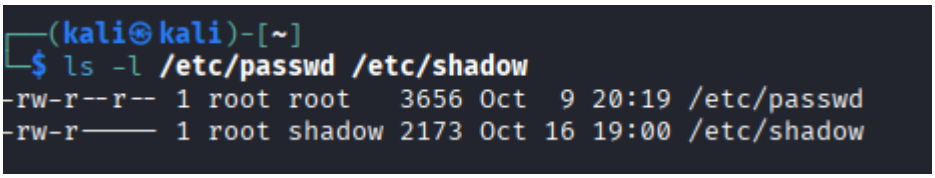


A2. Inspect shadow-file permissions

The /etc/shadow file stores password-hash fields and password-aging data. It is normally more restricted than /etc/passwd because its contents are security-sensitive.

```
ls -l /etc/passwd /etc/shadow
```

Expected result: Record the owner and permission bits shown by your VM.

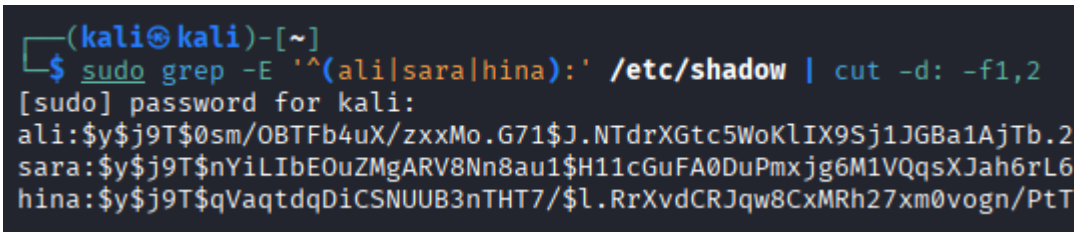


A3. Inspect password-hash fields (keep private)

This command displays the selected users' first two fields only. The output still contains sensitive hashes, so do not include the raw output in the report. Record only whether an entry exists and the hash-format prefix.

```
sudo grep -E '^(ali|sara|hina):' /etc/shadow | cut -d: -f1,2
```

Expected result: Entries appear only for accounts that exist. Do not paste the hash values into this report.



A4. Demonstrate a SHA-256 digest

This hashes the text 'hello' as a general checksum demonstration. SHA-256 alone is not an appropriate replacement for a purpose-built password hashing scheme.

```
echo -n 'hello' | sha256sum
```

Expected result: A hexadecimal digest is printed.

A5. Demonstrate Unix password-hash format

OpenSSL's -6 option generates a SHA-512-based Unix crypt password-hash format. It does not change a Linux account password.

```
openssl passwd -6 'LabDemo123!'
```

Expected result: The output normally starts with 6. Do not use a real password for this demonstration.

Task B — Salt and Hash Comparison

B1. Check that the lab accounts exist

Use the account database lookup to see whether the three expected lab users are present.

```
getent passwd ali sara hina
```

Expected result: Record which of the accounts are returned. If an account is missing, check with the instructor before creating or changing accounts.

B2. Confirm that a normal user cannot read shadow

This runs the read attempt as ali rather than as root. The expected result is a permission error on a normally configured system.

```
sudo -u ali cat /etc/shadow
```

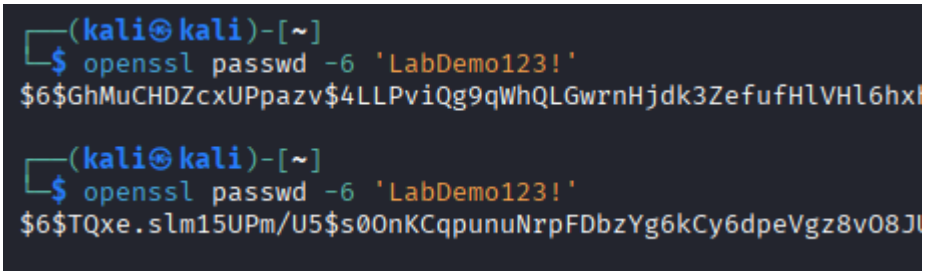
Expected result: Expected: permission denied. If access is granted, check the VM and file permissions with the instructor.

B3. Compare salted and unsalted outputs

Run the salted example twice. Random salts usually make the resulting strings different even when the input is the same.

```
openssl passwd -6 'LabDemo123!'
# Run the same command a second time
```

Expected result: Compare the two outputs locally; do not paste the full hashes into the report.



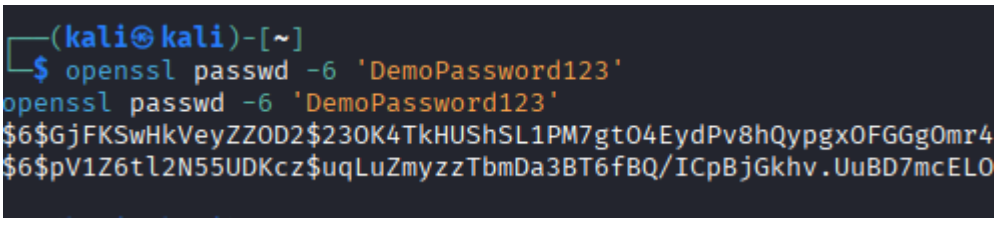
B4. Compare a legacy hash and a plain digest

The -1 option produces a legacy MD5-based Unix crypt format for demonstration only. The sha256sum command produces a deterministic digest for the same input.

```
openssl passwd -1 'LabDemo123!'
echo -n 'LabDemo123!' | sha256sum
```

Expected result: Do not use MD5 crypt or raw SHA-256 for real password storage. Record the difference between salted password hashes and a deterministic digest.

Observation to complete: The same password can produce different salted hashes because each hash can use a different salt. A plain SHA-256 digest of the same exact input remains the same.



Task C — Password Quality and Expiry

C1. Check PAM password-quality integration

This searches the common password PAM configuration for a reference to the pwquality module.

```
grep pwquality /etc/pam.d/common-password
```

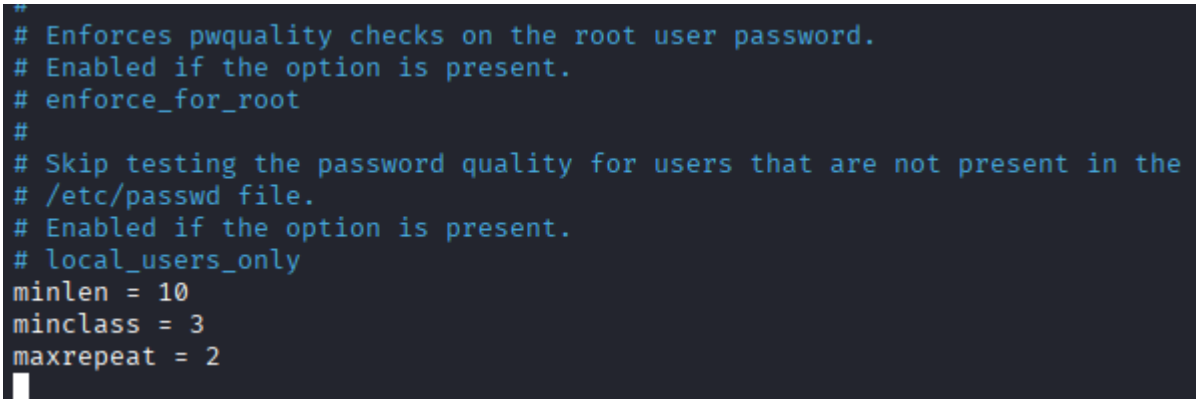
Expected result: Record whether a pam_pwquality.so line is present. If it is absent, ask the instructor before editing PAM.

C2. Review password-quality settings

The settings below represent the intended lab policy. Inspect the file first and avoid appending duplicate entries if they already exist.

```
sudo nano /etc/security/pwquality.conf
```

Expected result: Set or confirm: minlen = 10, minclass = 3, maxrepeat = 2. Save in nano with Ctrl+O, Enter, then Ctrl+X.



C3. Inspect and set password aging

The first command shows the current aging rules. The second sets a 90-day maximum age and a 7-day warning period for ali.

```
sudo chage -l ali
sudo chage -M 90 -W 7 ali
sudo chage -l ali
```

Expected result: Check that the maximum age and warning period match the requested values.

```
┌──(kali㉿kali)-[~]
└─$ sudo chage -l ali
Last password change                                    : Oct 09, 202
Password expires                                        : never
Password inactive                                       : never
Account expires                                         : never
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7

┌──(kali㉿kali)-[~]
└─$ sudo chage -M 90 -W 7 ali

┌──(kali㉿kali)-[~]
└─$ sudo chage -l ali
Last password change                                    : Oct 09, 202
Password expires                                        : Jan 07, 202
Password inactive                                       : never
Account expires                                         : never
Minimum number of days between password change          : 0
Maximum number of days between password change          : 90
Number of days of warning before password expires       : 7
```

C4. Mark the password expired for the exercise

This marks ali's password as expired so that a password-change prompt can be tested at the next suitable login.

```
sudo chage -d 0 ali
```

Expected result: Confirm the password-aging status with sudo chage -l ali. Only do this in the lab VM.

C5. Test password-quality enforcement

Switch to the lab account and run passwd. Try the deliberately weak examples only if permitted by the lab, note the actual rejection messages, and then set an acceptable lab password.

```
sudo -iu ali
passwd
```

Expected result: Record the exact messages shown by your VM. Password input is not echoed to the terminal. Exit the account with exit when finished.

```
┌──(ali㉿kali)-[~]
└─$ passwd
Changing password for ali.
Current password:
New password:
Retype new password:
The password has not been changed.
New password:
Retype new password:
You must choose a longer password.
New password:
Retype new password:
passwd: password updated successfully

┌──(ali㉿kali)-[~]
```

Task D — TOTP and PAM

D1. Check the VM clock and back up PAM

TOTP depends on time. Verify the clock, then create a backup before changing the su PAM configuration.

```
date
sudo cp /etc/pam.d/su /root/su.pam.orig
```

Expected result: The copy command normally produces no output when successful.

D2. Enrol the lab user for TOTP

Run the authenticator setup as the lab user. Follow its prompts and add the QR code or secret to an authenticator app. Keep the secret and emergency codes private.

```
sudo -iu ali
google-authenticator -t -d -f -r 3 -R 30 -w 3
```

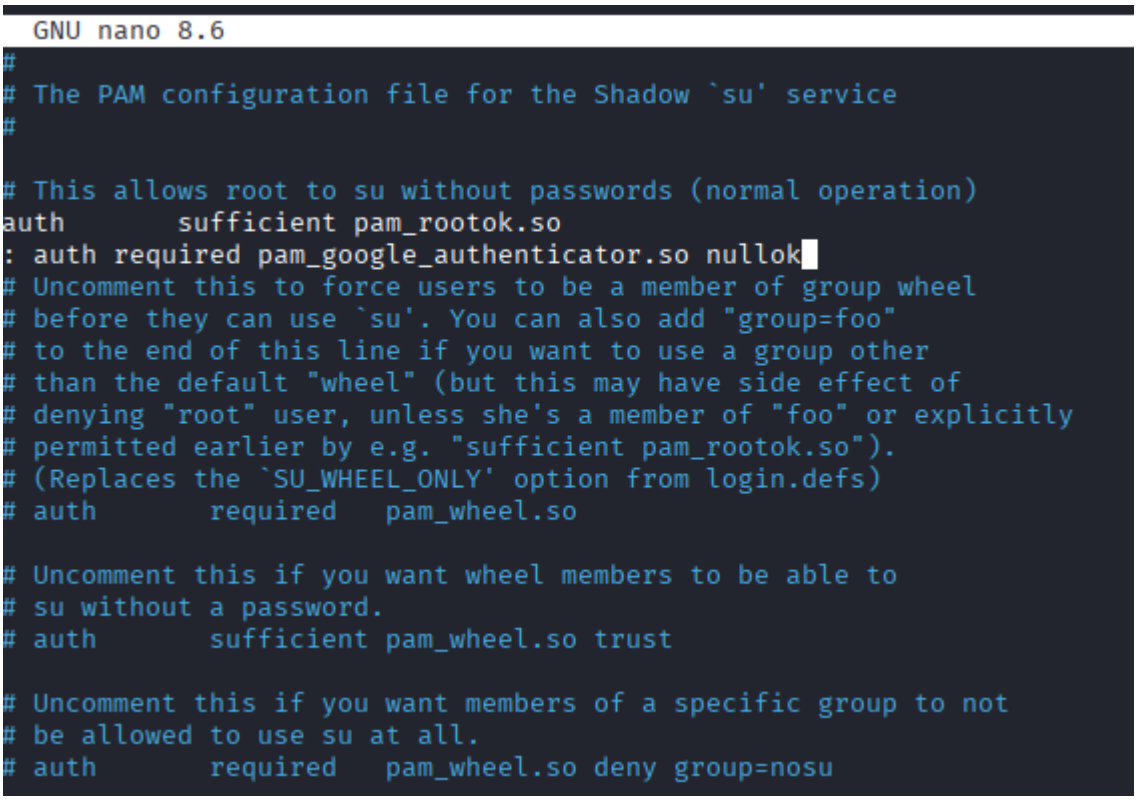
Expected result: Record whether setup completed. Do not capture or submit the QR code, secret key, or emergency codes.

D3. Add the TOTP PAM module for su

Open the su PAM file and add the module line immediately after the common-auth include, following the lab manual. Do not change unrelated PAM files.

```
sudo nano /etc/pam.d/su
```

Expected result: Line to add: auth required pam_google_authenticator.so nullok. Save with Ctrl+O, Enter, Ctrl+X. The nullok option may allow accounts without an enrolled secret to pass this module, depending on configuration.



Task E — Recovery and Rollback

E1. Test an emergency code

Use one saved emergency code in place of a TOTP code. Then attempt to reuse the same code. Emergency codes are normally single-use.

```
su - ali
```

Expected result: Record whether the first use succeeded and whether reuse was rejected. Never include the emergency code itself.

E2. Reset TOTP enrollment

This removes the lab user's authenticator secret file. It is destructive and should be used only for the reset step in the isolated lab VM.

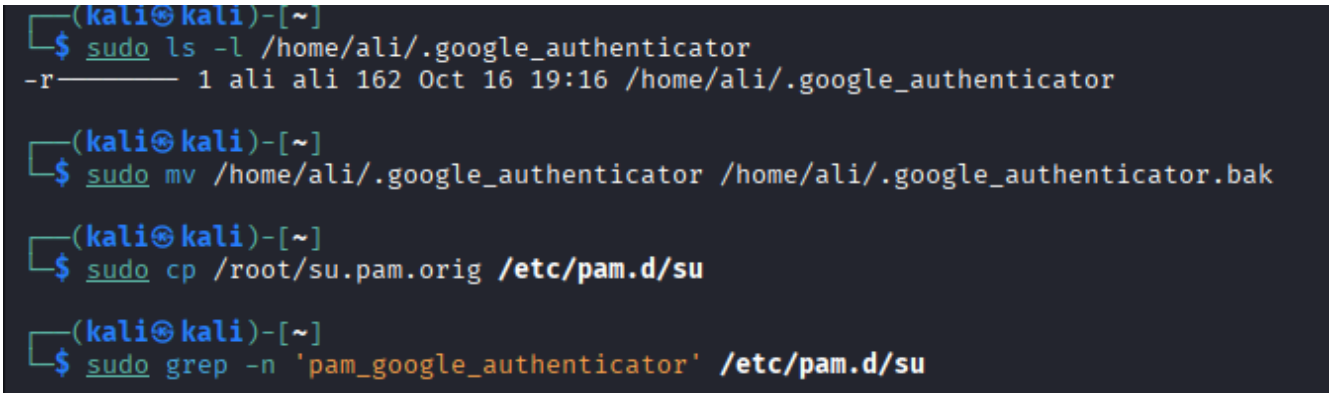
```
sudo rm /home/ali/.google_authenticator
```

Expected result: The file is removed if the path exists and permissions allow it. Confirm the intended account and path before running.

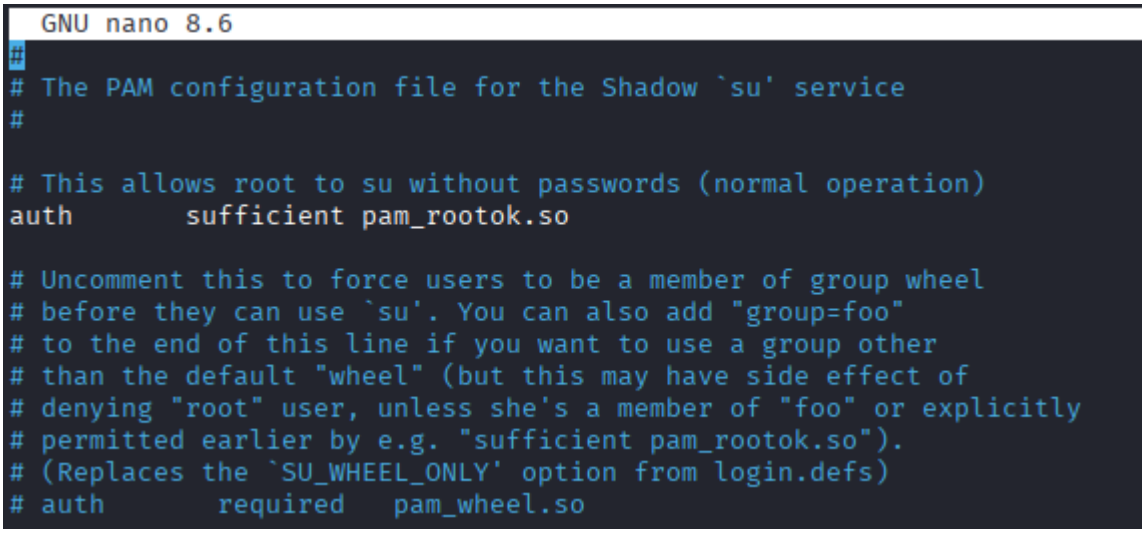
E3. Restore the original PAM configuration

Restore the backup created before the PAM edit, then check whether the Google Authenticator line remains.

```
sudo cp /root/su.pam.orig /etc/pam.d/su
grep google /etc/pam.d/su
```



Expected result: If the original file did not contain the module line, grep should return no match. Confirm the backup exists before restoring.


GNU nano 8.6
#
The PAM configuration file for the Shadow `su' service
#
This allows root to su without passwords (normal operation)
auth sufficient pam_rootok.so
Uncomment this to force users to be a member of group wheel
before they can use `su'. You can also add "group=foo"
to the end of this line if you want to use a group other
than the default "wheel" (but this may have side effect of
denying "root" user, unless she's a member of "foo" or explicitly
permitted earlier by e.g. "sufficient pam_rootok.so").
(Replaces the `SU_WHEEL_ONLY' option from login.defs)
auth required pam_wheel.so